《算法设计与分析》实验报告

实验七　综合实验

报告书			
姓名	顾凯琪	指导教师	方贤忠
学号	226001624	日　　期	20250617
班级	22 计算机 2 班		

实验内容

[OJ 测试]

实验目的

学习掌握本类算法的设计与实现，完成实验报告书写。

【如有查重雷同，请在此说明】

代码独立完成，无雷同。

【如有 OJ 未完成，请在此说明，并在本实验文档中补全】

都已完成

实验过程和步骤

花花草草

Description

有一个M*M的草坪，周边被篱笆围着。数值1表示该区域种着花，数值0表示该区域是草地。如果花的上下左右也是花，则这些连着的花必须是同一种花，不相连的花的品种不同。请你计算某个给定草坪需要几种花。

Input

第一行一个整数M，表示草坪的大小；

后面是M行M列的整数，表示草坪上花和草的分布情况。

Output

一个整数，表示所需要的花的品种数量。

Sample Input 1

```
3
0 0 0
0 0 0
0 0 0
```

Sample Output 1

```
0
```

```
1. #include<bits/stdc++.h>
2. using namespace std;
```

```
3.
4. int M;
5. vector<vector<int>> place;
6. vector<vector<bool>> visited;
7. int dir[4][2] = {{-1,0},{1,0},{0,-1},{0,1}};
8. queue<pair<int,int>> qu;
9.
10.void bfs(int x,int y){
11. visited[x][y] = true;
12. qu.push({x,y});
13.
14. while(!qu.empty()){
15.  pair<int,int> temp = qu.front();
16.  qu.pop();
17.
18.  for(int i =0;i<4;i++){
19.   int newx = temp.first + dir[i][0];
20.   int newy = temp.second + dir[i][1];
21.
22.   if(newx>=0 &&newx <M &&newy >=0 &&newy <M &&visited[newx][ne
   wy] == false && place[newx][newy] == 1){
23.    qu.push({newx,newy});
24.    visited[newx][newy] = true;
25.   }
26.  }
27. }
28.}
29.
30.int main(){
31. cin>>M;
32. place = vector<vector<int>>(M, vector<int>(M));
33.    visited = vector<vector<bool>>(M, vector<bool>(M, false));
34. int count = 0;
35. for(int i = 0;i<M;i++){
36.  for(int j =0;j<M;j++){
37.   cin>>place[i][j];
38.   visited[i][j] = false;
39.  }
40. }
41. for(int i = 0;i<M;i++){
42.  for(int j =0;j<M;j++){
43.   if(place[i][j] == 1 && visited[i][j] == false){
44.    bfs(i,j);
```

```
    count++;
   }
  }
 }
 cout<<count<<endl;

}
```

临时抱佛脚

Description

到学期末了，小K同学决定开始学习。由于时间紧迫，他决定合理规划时间，以期取得最好的成绩。小K同学有M天的时间复习N门课程，每门课程花费不同的天数，会有不同的预期成绩。小K需要合理安排这M天，使得总的预期成绩最大。

Input

整数N（课程数<100）和整数M（天数<100）；

后面是i=1..N行数据，每行j=1..M个数据；第i行数据表示第i门课，花费j天的时间获得的预期成绩。

Output

一个整数，表示最大预期总成绩。

Sample Input 1

```
2 2
1 2
1 3
```

Sample Output 1

```
3
```

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int N, M;
    cin >> N >> M;

    vector<vector<int>> score(N + 1, vector<int>(M + 1, 0));
    for (int i = 1; i <= N; i++) {
        for (int j = 1; j <= M; j++) {
            cin >> score[i][j];
        }
    }

    vector<int> dp(M + 1, 0);

    for (int i = 1; i <= N; i++) {
        for (int j = M; j >= 0; j--) {
            for (int k = 1; k <= j; k++) {
                dp[j] = max(dp[j], dp[j - k] + score[i][k]);
            }
        }
    }

```

```
    cout << dp[M] << endl;
    return 0;
}
```

森林迷宫

Description

一天Luna在森林里探险的时候不小心走入了一个迷宫，迷宫可以看成是由n*n的格点组成，每个格点只有2种状态：^ 和 # ； 前者表示可以通行后者表示不能通行。当Luna处在某个格点时，她动到东南西北(或者说上下左右)四个方向之一的相邻格点上，Luna想要从起点A走到终点B（中途不能走出迷宫）。如果起点或者终点有一个不能通行(为#)，则当做无法通行。

Input

第1行是测试数据的组数k，后面跟着k组输入。

每组测试数据的第1行是一个正整数n (1 <= n <= 100)，表示迷宫的规模是n*n的。

接下来是一个n*n的矩阵，矩阵中的元素为 ^或者 #。

再接下来一行是4个整数ar,ac,br,bc。表示A处在第ar行，第ac列，B处在第br行, 第bc列。注意坐标ar，ac，br，bc全部是从0开始计数的.

Output

YES或NO

Sample Input 1

```
2
3
^##
^^#
#^^
0 0 2 2
5
^^^^^
###^#
^^#^^
###^^
^^^#^
0 0 4 0
```

Sample Output 1

```
YES
NO
```

```
#include<bits/stdc++.h>
using namespace std;

int n,ar,ac,br,bc;
char place[101][101];
int dir[4][2] = {{-1,0},{1,0},{0,-1},{0,1}};
bool visited[101][101] = {false};
bool flag = false;

void dfs(int x,int y){
 if(flag) return;
 if(visited[x][y]) return;
 if(x == br &&y == bc){
  cout<<"YES"<<endl;
  flag = true;
  return;
 }

 visited[x][y] = true;
 for(int i =0;i<4;i++){
   int newx = x + dir[i][0];
   int newy = y + dir[i][1];

   if(newx>=0 &&newx <n &&newy >=0 &&newy <n&& place[newx][newy
```

```
   ] == '^' &&flag == false){
25.     dfs(newx,newy);
26.    }
27.   }
28. //visited[x][y] = false;
29.}
30.
31.int main(){
32. int k;
33. cin>>k;
34. while(k--){
35.  flag = false;
36.
37.  cin>>n;
38.  for(int i =0;i<n;i++){
39.   for(int j = 0;j<n;j++){
40.    cin>>place[i][j];
41.    visited[i][j] = false;
42.   }
43.  }
44.  cin>>ar>>ac>>br>>bc;
45.  if(place[ar][ac] == '^' && place[br][bc] == '^'){
46.   dfs(ar, ac);
47.  }
48.   if(!flag){
49.      cout<<"NO"<<endl;
50.     }
51. }
52.}
```

减肥

Description

小K是个苦命的孩子，他的师傅为了多赚钱，以减肥为理由，让他去采药，并说不完成不能吃饭。野地里有许多不同的草药，采每一株都需要一些时间，每一株也有它自身的价值。要求在规
间里，采到的草药的总价值最大。

自从小K成功的完成了前面的采药任务后，师傅还觉得小K可以更瘦点，省点吃饭的成本，于是给小K换了一个地方：每种草药可以无限制地疯狂采摘。

Input

第一行有2个整数T(1≤T≤100000)和M(1≤M≤100)，一个空格隔开，T代表总共能够用来采药的时间，M代表山洞里的草药的数目。

接下来的M行每行包括两个在1到100之间包括1和100）的整数，分别表示采摘某株草药的时间和这株草药的价值。

Output

1个整数，表示在规定的时间内可以采到的草药的最大总价值。行末有换行。

Sample Input 1

```
70 3
71 100
69 1
1 2
```

Sample Output 1

```
140
```

```
1.
#include <bits/stdc++.h>
```

```cpp
using namespace std;
int dp[100005];
int main(){
    int T,M;
    if(!(cin>>T>>M)) return 0;
    int ti, vi;
    while(M--){
        cin>>ti>>vi;
        for(int t=ti; t<=T; t++){
            dp[t] = max(dp[t], dp[t-ti] + vi);
        }
    }
    cout<<dp[T]<<'\n';
    return 0;
}
```



不挂科

Description

n个同学顺序围成一个圈，传递一个"不挂科"令牌，如果最后一次能传回第一个同学，则得到过的人不会挂科；每个同学只能传给左边或者右边的同学；现在从1号同学开始，第1次传递到1；
以从1传递到2或n。问经过m次传递后，传回到1号同学的可能方式是多少种?

Input

两个整数m,n；（1<=m<=100；1<=n<=100）。

Output

共有多少种传法。

Sample Input 1

```
9 8
```

Sample Output 1

```
72
```

```cpp
#include <iostream>
using namespace std;

int main() {
    int m, n;
    cin >> m >> n;

    int dp[101][101] = {0};
    dp[1][1] = 1;

    for (int i = 2; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            if (j == 1)
                dp[i][j] = dp[i - 1][2] + dp[i - 1][n];
            else if (j == n)
                dp[i][j] = dp[i - 1][1] + dp[i - 1][n - 1];
            else
                dp[i][j] = dp[i - 1][j - 1] + dp[i - 1][j + 1];
```

```
        }
    }

    cout << dp[m][1] << "\n";
    return 0;
}
```

最佳调度

Description

假设有 n 个任务由 k 个可并行工作的机器完成。完成任务 i 需要的时间为t_i。试设计一个算法找出完成这 n 个任务的最佳调度，使得完成全部任务的时间最早。对任意给定的整数 n 和 k，以及任务i需要的时间为t_i，i=1~n。设计一个优先队列式分支限界法，计算完成这 n 个任务的最佳调度。如果没有按照题目要求用分支限界法解题，则所得分数减半。

Input

第一行有 2 个正整数 n 和 k，$n \leq 20$。第 2 行的 n 个正整数是完成n个任务需要的时间。

Output

将计算出的完成全部任务的最早时间输出.

Sample Input 1

```
7 3
2 14 4 16 6 5 3
```

Sample Output 1

```
17
```

```
#include<bits/stdc++.h>
using namespace std;

struct Node {
    vector<int> load;
    int task;
    int maxLoad;
    bool operator<(const Node &b) const {
        return maxLoad > b.maxLoad;
    }
};

int main() {
    int n, k;
    cin >> n >> k;
    vector<int> t(n);
    for(int i = 0; i < n; i++) cin >> t[i];

    sort(t.rbegin(), t.rend());

    priority_queue<Node> q;
    Node start;
    start.load = vector<int>(k, 0);
    start.task = 0;
    start.maxLoad = 0;
    q.push(start);
```

```
27.
28.     int best = accumulate(t.begin(), t.end(), 0);
29.
30.     while(!q.empty()) {
31.         Node cur = q.top(); q.pop();
32.         if(cur.task == n) {
33.             best = min(best, cur.maxLoad);
34.             continue;
35.         }
36.         for(int i = 0; i < k; i++) {
37.             Node next = cur;
38.             next.load[i] += t[cur.task];
39.             next.task++;
40.             next.maxLoad = *max_element(next.load.begin(), next
    .load.end());
41.             if(next.maxLoad >= best) continue;
42.             q.push(next);
43.         }
44.     }
45.
46.     cout << best <<endl;
47.     return 0;
48. }
```